Laborator 4 - LFA

Mocanu Ștefan

1 Conversia între λ-NFA și Expresii Regulate (RegEx)

Expresiile regulate și automatele finite sunt echivalente ca putere de calcul (Teorema lui Kleene). În acest laborator, ne concentrăm pe algoritmii de trecere dintr-o formă în alta.

1.1 Algoritmul de transformare: λ-NFA $\rightarrow$ RegEx

Pentru a extrage o expresie regulată dintr-un automat, utilizăm metoda eliminării stărilor. Pașii sunt următorii:

1. **Standardizarea automatului:**
   - Se adaugă o nouă stare de start q_s și o tranziție λ de la q_s la vechea stare inițială q_0.
   - Se adaugă o nouă stare finală unică q_f și tranziții λ de la toate vechile stări finale F către q_f.
2. **Precalcularea (Unificarea tranzițiilor):** Înainte de a începe eliminarea stărilor, trebuie să ne asigurăm că între oricare două stări q_i și q_j există **cel mult o tranziție**.
   - Dacă există mai multe tranziții de la q_i la q_j etichetate cu simbolurile $\{a_1, a_2, \ldots, a_n\}$, acestea se înlocuiesc cu o singură tranziție etichetată cu expresia reunită: $(a_1|a_2|\ldots|a_n)$.
3. **Eliminarea succesivă a stărilor:** Pentru fiecare stare intermediară $q \in Q$ (altele decât q_s și q_f), se elimină starea prin actualizarea etichetelor de pe tranzițiile între vecinii săi.
4. **Regula de abstractizare:** Dacă avem o stare k pe care dorim să o eliminăm, aflată între stările p și r, actualizăm tranziția directă $p \rightarrow r$ folosind formula:

   $$R_{new} = R_{pr} \cup (R_{pk} \cdot (R_{kk})^* \cdot R_{kr})$$

   unde:
   - R_{pr} este expresia curentă între p și r.
   - R_{pk} este expresia de la p la k.
   - R_{kk} este auto-bucla stării k.
   - R_{kr} este expresia de la k la r.

   Nota: Este posibil ca intre p și r să nu existe tranzitie, asa că formula se reduce la:

   $$R_{new} = R_{pk} \cdot (R_{kk})^* \cdot R_{kr}$$

1.2 Algoritmul de transformare: RegEx $\rightarrow$ λ-NFA

Pentru conversia inversă, utilizăm **Construcția Thompson**. Deoarece expresiile pot fi complexe, procesul presupune două etape:

1.2.1 Sortarea operatorilor (Shunting Yard)

Pentru a gestiona precedența operatorilor, transformăm expresia din notație infixată în notație postfixată (Forma Poloneză Inversă) folosind algoritmul **Shunting Yard**.

1.2.2 Preprocesarea: Inserarea concatenării

Deoarece în RegEx concatenarea este adesea implicită (ab în loc de $a.b$), trebuie să inserăm un simbol special (vom folosi ".") între:

- Un operand și o paranteză deschisă: $a(\to a.($
- O paranteză închisă și un operand: $)a \to).a$
- Doi operanzi: $ab \to a.b$
- Un operator unar ($*, +$) și un operand sau paranteză deschisă: $a^*b \to a^*.b$

1.2.3 Pseudocod Shunting Yard

Input: Expresia regulată infixată (ex: `a.(b|c)*`)
Output: Coada de ieșire în formă postfixată

1. **Pentru fiecare** simbol s din expresie:
   - **Dacă** s este literă: Adaugă s la coada de ieșire.
   - **Dacă** s este '(': Pune s în stiva de operatori.
   - **Dacă** s este ')': Extrage operatorii din stivă și adaugă-i la ieșire până când găsești '('. Elimină '('.
   - **Dacă** s este operator ($*, +, |, .$):
     - Cât timp (stiva nu e goală **și** prioritatea lui $s \leq$ prioritatea operatorului din vârful stivei):
       - Extrage din stivă în coada de ieșire.
     - Pune s în stiva de operatori.
2. **La final:** Extrage toți operatorii rămași în stivă și adaugă-i la ieșire.

1.2.4 Tabela de priorități

În implementare, puteți folosi o funcție simplă de tip helper pentru a returna prioritatea:

Operator	Prioritate	Tip
(	4	Paranteză (Prioritate maximă)
$*, +$	3	Unar
. (concatenare)	2	Binar
| (reuniune)	1	Binar (Prioritate minimă)

1.2.5 Construcția recursivă

Folosind stiva rezultată din Shunting Yard, construim fragmente de automat pentru fiecare operator și le combinăm folosind λ-tranziții conform regulilor standard (ex: pentru $A|B$ creăm o ramificație, pentru A^* creăm o buclă cu λ).

Odată obținută forma postfixată, construcția automatului se face folosind o **stivă de automate**:

- **Operand (literă):** Se creează un automat cu două stări și o tranziție etichetată cu acea literă. Se introduce automatul în stivă.
- **Operator Binar ($|, .$):**
  1. Se extrag două automate din stivă: A_2 (ultimul) și A_1 (penultimul).
  2. Se combină conform regulii (ex: pentru | se creează noi stări de start și final și se leagă cu λ de vechile intrări/ieșiri).
  3. Noul automat rezultat se introduce înapoi în stivă.
- **Operator Unar ($*, +$):**
  1. Se extrage un singur automat A din stivă.
  2. Se aplică transformarea Kleene (adăugarea buclelor λ).
  3. Se introduce înapoi în stivă.

2 Automate Push-down (PDA)

Automatele Push-down extind capabilitățile NFA prin adăugarea unei stive, permițând recunoașterea limbajelor independente de context (ex: paranteze echilibrate $a^n b^n$).

Un PDA este definit de tuplul $(Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$, unde:

- Γ este alfabetul stivei.
- Z_0 este simbolul inițial de pe stivă.
- $\delta : Q \times (\Sigma \cup \{\lambda\}) \times \Gamma \to \mathcal{P}(Q \times \Gamma^*)$ este funcția de tranziție.

2.1 Anatomia unei tranziții PDA

Spre deosebire de un DFA/NFA, unde o tranziție depinde doar de simbolul citit, la un Automat Push-down (λ-PDA) o tranziție este determinată de un triplet și produce un set de acțiuni.

O tranziție se scrie sub forma:

$$\delta(q, a, Z) = \{(p, \gamma)\}$$

Semnificația componentelor este următoarea:

- **Input (Ce trebuie să avem):**
  - $q \in Q$: Starea curentă.
  - $a \in \Sigma \cup \{\lambda\}$: Simbolul citit din șirul de intrare (poate fi λ dacă nu consumăm input).
  - $Z \in \Gamma$: Simbolul aflat în **vârful stivei** (trebuie extras/verificat).
- **Output (Ce rezultă):**
  - $p \in Q$: Noua stare în care trece automatul.
  - $\gamma \in \Gamma^*$: Un șir de simboluri care se pun pe stivă în locul lui Z.

2.1.1 Operații asupra stivei în timpul tranziției

În funcție de valoarea lui γ, avem trei operații fundamentale:

1. **POP** (Eliminare): $\gamma = \lambda$.
   Simbolul Z este scos de pe stivă și nu se pune nimic în loc.
2. **SKIP/REPLACE** (Modificare): $\gamma = Y$.
   Simbolul Z este înlocuit cu Y. Dacă $Y = Z$, stiva rămâne neschimbată la vârf.
3. **PUSH** (Adăugare): $\gamma = YZ$ sau $\gamma = Y_1 Y_2 \ldots Z$.
   Simbolul Z este înlocuit cu un șir de simboluri (de obicei se pune noul simbol Y peste vechiul Z). *Atenție: Ordinea în γ contează (primul simbol din γ devine noul vârf).*

2.1.2 Condiții de acceptare

Există două moduri standard prin care un PDA poate accepta un cuvânt:

- **Prin stare finală:** Automatul se află într-o stare $q \in F$ după consumarea întregului cuvânt.
- **Prin stivă vidă:** Automatul a golit complet stiva după consumarea întregului cuvânt.
- **Prin ambele conditii:** Automatul trebuie sa indeplineasca ambele conditii descrise.

3 Temă de laborator

1. **Obligatoriu:** Implementați un program care transformă un λ-NFA dat într-o Expresie Regulată echivalentă.
2. **Obligatoriu:** Implementați un simulator de Automat Push-down (PDA) care primește definiția automatului și un șir de intrare, determinând dacă șirul este acceptat, in definitia acestuia intra si conditia de acceptare.
3. **Bonus (Inversare):** Implementați transformarea unei Expresii Regulate în λ-NFA (necesită implementarea Shunting Yard pentru parsarea string-ului).

Notă tehnica: Rezultatul transformării RegEx $\to$ λ-NFA poate fi utilizat ca input pentru algoritmul de transformare în DFA din laboratorul anterior pentru a obține un automat minimal.